Formação Completa

AWS Developer Associate



Domínio 3: Implantação na AWS

Bem-vindos ao Domínio 3: Implantação (Deployment) na AWS.

Tem como objetico capacitar o desenvolvedor para entregar código funcional na nuvem, garantindo boas práticas de CI/CD, uso de infraestrutura como código, organização dos artefatos de aplicação, e a correta separação de ambientes.

Preparando Artefatos e Empacotamento na AWS

Objetivos

Gerenciamento de Configurações e Segredos

AppConfig e Secrets Manager

Empacotamento de Aplicações

Lambda (ZIP/Layers) e Contêineres

Versionamento e Repositórios

Git e integração com CI/CD

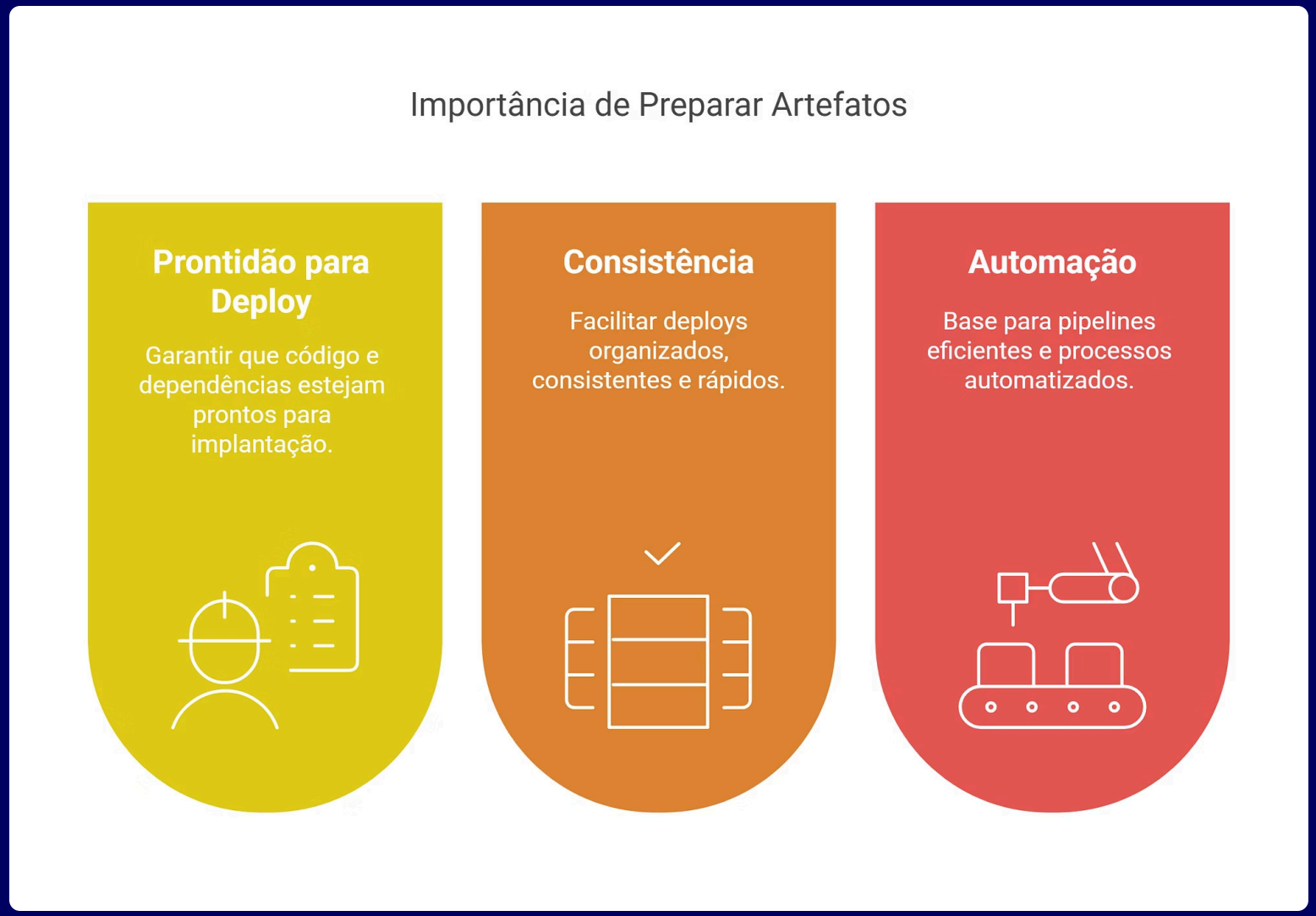
Automação CI/CD

CodePipeline, CodeBuild, CodeDeploy

Infraestrutura como Código

CloudFormation

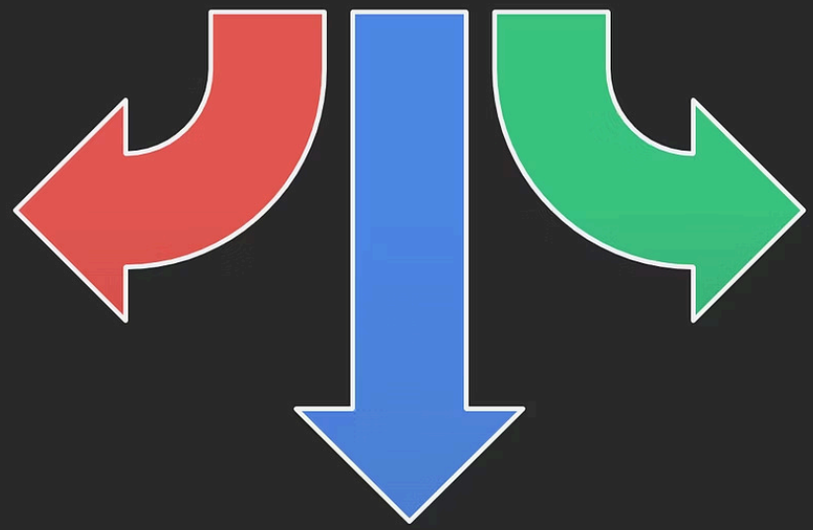
Por que Preparar Artefatos é Crucial?



O Desafio das Configurações

Segredos Sensíveis

Garante que informações confidenciais não sejam expostas no código.



Feature Flags

Permite atualizações dinâmicas sem redeploy.

Múltiplos Ambientes

Adapta configurações para diferentes estágios de desenvolvimento.



AWS AppConfig

O que é?

Serviço para gerenciar configurações de aplicações de forma centralizada e dinâmica.

Parte do AWS Systems Manager.

Permite atualizações sem novo deploy.



Benefícios e Uso do AppConfig

Atualizações Dinâmicas

Feature flags e ajustes em tempo real.

Adaptação Rápida

Responda rapidamente a mudanças de requisitos.

Centralização

Manutenção ágil e redução de erros.

Uso via SDK

Recupere configurações facilmente com Boto3.

Exemplo de AppConfig em Python

```python
import boto3

appconfig = boto3.client('appconfig', region_name='us-east-1')

response = appconfig.get_configuration(
    Application='MyApplication',
    Environment='Production',
    Configuration='MyConfiguration',
    ClientId='UniqueClientId'
)

config_data = response.get('Content').read().decode()
```

O código recupera configurações do AppConfig usando o SDK Boto3.

A configuração é geralmente definida via Console, CLI ou IaC.

O ClientId identifica unicamente o cliente que solicita a configuração.

A Necessidade de Empacotamento

Organização

Como organizar código e dependências para deploy eficiente?

Objetivo

Processo simples, rápido e consistente.

Abordagens

Diferentes métodos para Lambda vs. Contêineres.



Empacotando Funções Lambda

Método 1: Arquivo ZIP

- Código da função + dependências
- Simples para funções pequenas

Método 2: Lambda Layers

- Separa dependências comuns em pacotes
- Reduz tamanho do pacote da função
- Facilita compartilhamento de bibliotecas

Exemplo de Deploy com AWS CLI

```
# Criar pacote zip com o código
zip function.zip my_lambda_function.py

# Fazer upload do código para a função Lambda
aws lambda update-function-code \
  --function-name MinhaFuncaoLambda \
  --zip-file fileb://function.zip
```

Esse processo também pode ser incluído em scripts de automação CI/CD.



Empacotando Contêineres

Docker

Tecnologia padrão para containerização.

Dockerfile

Define como construir a imagem.

Imagem Docker

Artefato final, autocontido e portável.

Amazon ECR

Repositório para armazenar imagens.

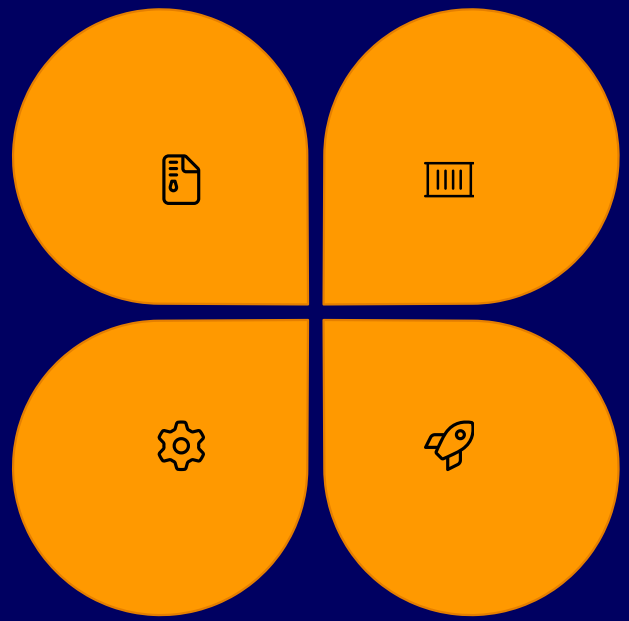
Pontos Chave do Empacotamento

Lambda

ZIP ou Layers para separar dependências

Contêineres

Imagem Docker armazenada no ECR



Automação

Requer processos de build automatizados

Objetivo

Deploys rápidos e consistentes

Repositórios de Código e Versionamento

A Base do Desenvolvimento

Gerenciar e controlar alterações no código-fonte.

Ferramenta padrão: Git.

Essencial para colaboração e rastreabilidade.



Benefícios do Git

Colaboração

Múltiplos desenvolvedores trabalhando juntos através de branches e merges.

Histórico

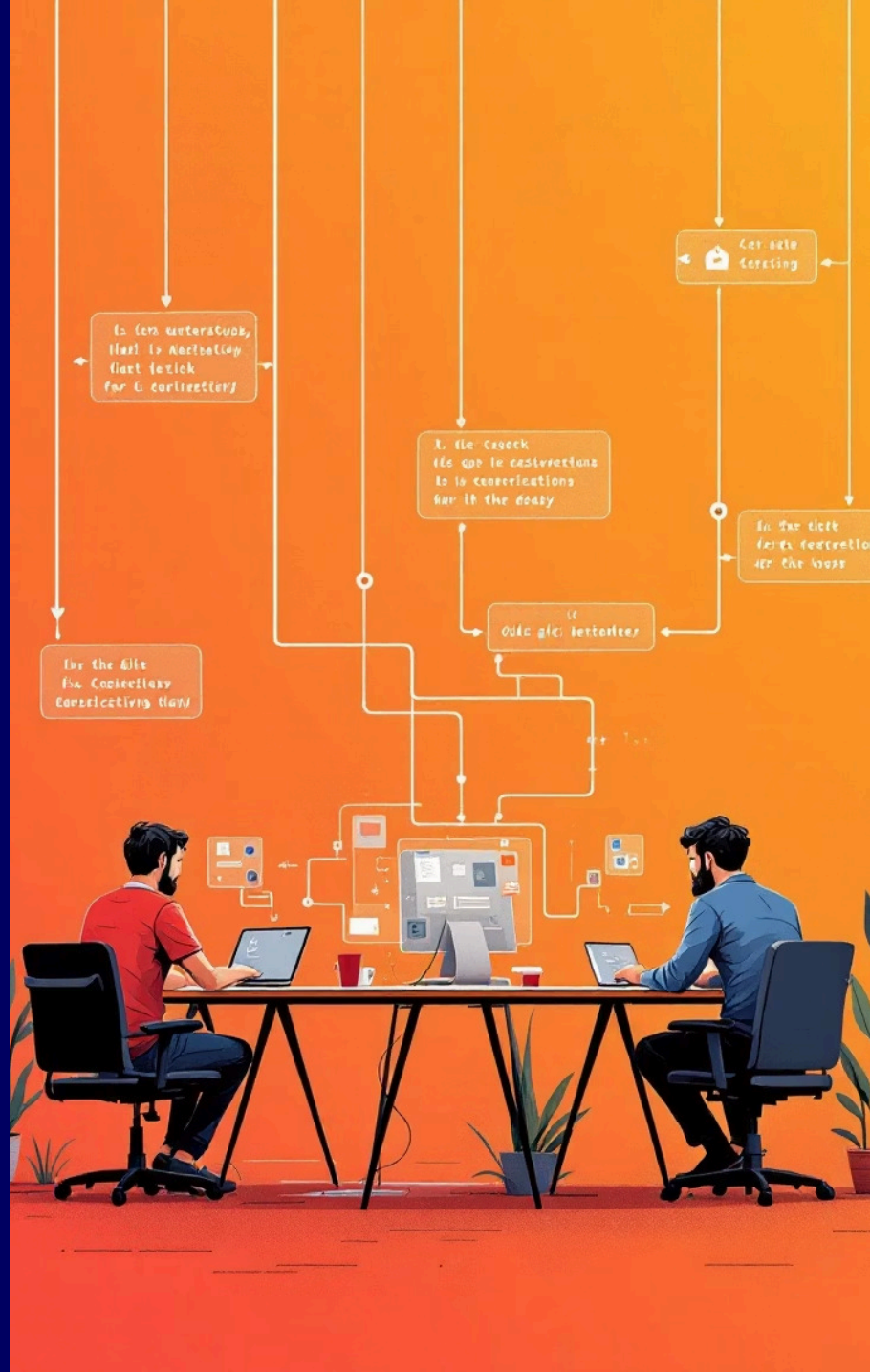
Registro detalhado de quem mudou o quê e quando.

Rollback

Facilidade para reverter para versões anteriores quando necessário.

Branching

Trabalhar em features ou correções isoladamente sem afetar o código principal.



Boas Práticas e Integração CI/CD



Commits Frequentes

Mensagens descritivas e mudanças pequenas.



Estratégias de Branch

Gitflow ou GitHub Flow para organização.



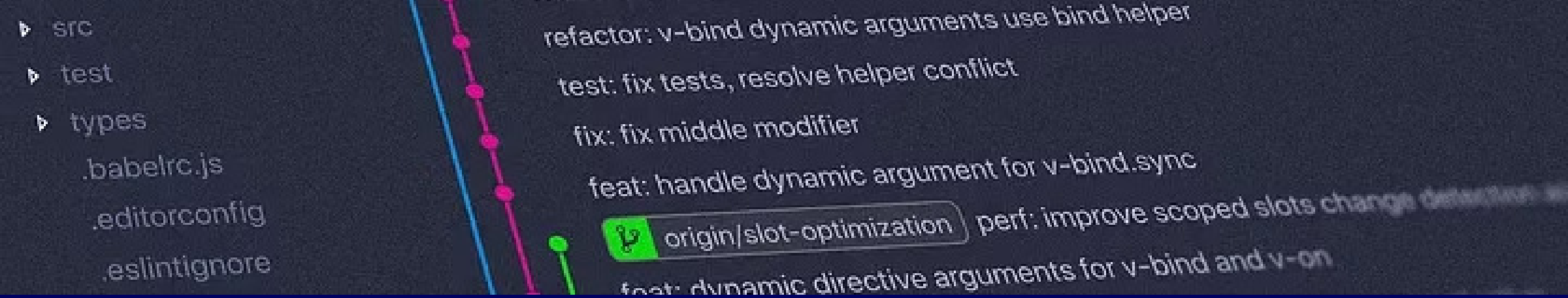
Code Reviews

Revisão por pares antes de integrar mudanças.



Integração CI/CD

Pipelines acionados por eventos do repositório.



Pontos Chave do Versionamento

Colaboração

Facilita trabalho em equipe e controle de versões.

Histórico

Registro detalhado e capacidade de rollback.

Automação

Integração com CI/CD para builds, testes e deploys.

Práticas

Requer disciplina em branching e reviews.

Automação com CI/CD na AWS

CI/CD

Integração Contínua (CI) e Entrega Contínua (CD).

Automatiza o ciclo: Build (construir) -> Test (testar) -> Deploy (implantar).

Serviços AWS

- CodeCommit (Git)
- CodeArtifact (Ger. de dependências)
- CodeBuild
- CodeDeploy
- CodePipeline



AWS CodePipeline - O Orquestrador

Define o Fluxo

Visualiza e gerencia o pipeline completo de entrega.

Conecta Serviços

Integra fontes de código, build, teste e deploy.

Orquestra Execução

Coordena a execução das etapas em sequência.

AWS CodeBuild - O Construtor

Serviço Gerenciado

Build totalmente gerenciado sem servidores para manter.

Compilação

Compila código-fonte conforme especificações do **buildspec.yml**.

Testes

Executa testes unitários e de integração automatizados.

Artefatos

Produz pacotes de deploy como ZIPs ou imagens Docker.





AWS CodeDeploy - O Implantador

Automação

Automatiza a implantação de aplicações em diversos ambientes.

Suporte

Funciona com EC2, ECS, Lambda e servidores on-premises.

Downtime

Minimiza tempo de inatividade durante atualizações.

Estratégias

Suporta In-Place, Blue/Green e Canary deployments.

Integração dos Serviços CI/CD

Repositório

Commit no Git inicia o processo

CodePipeline

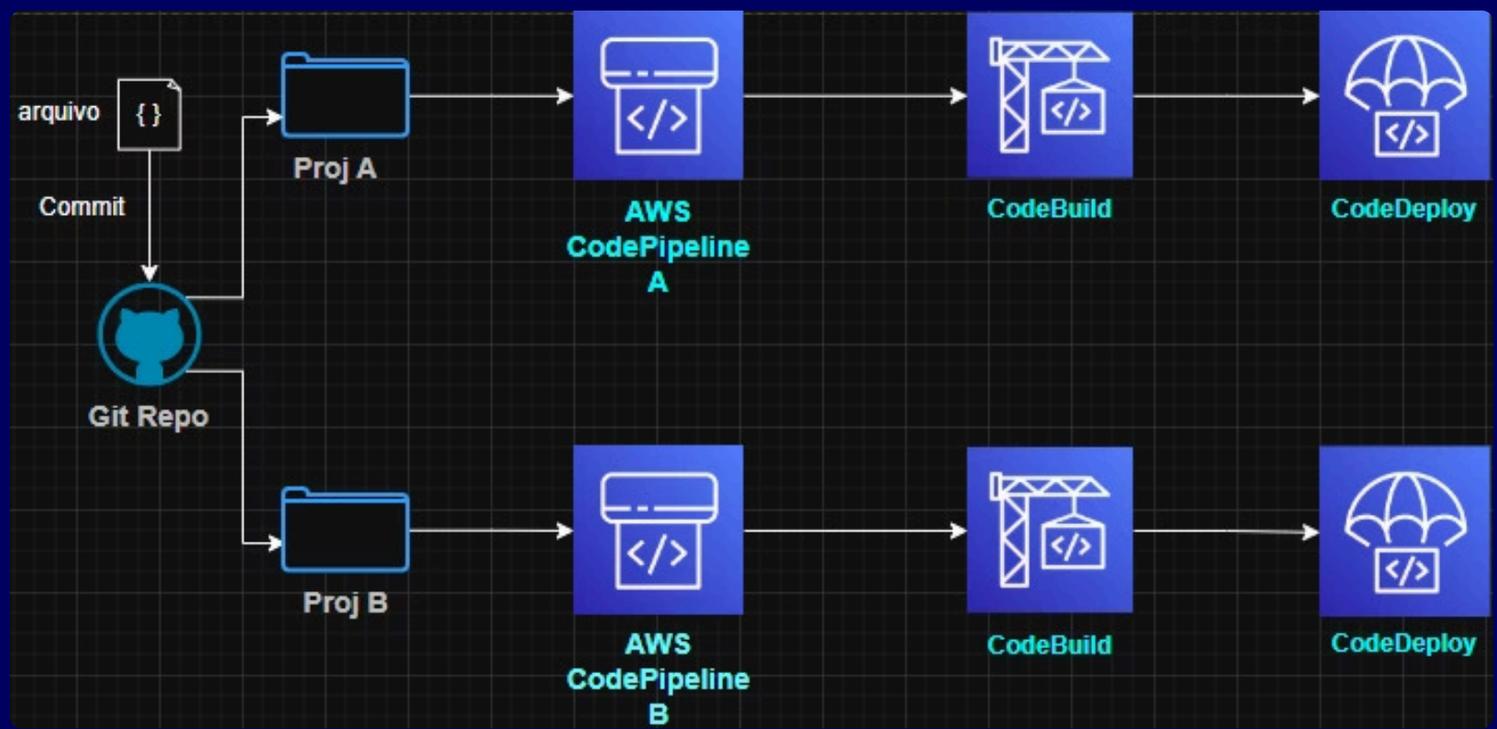
Detecta mudança e inicia pipeline

CodeBuild

Compila e testa, gerando artefato

CodeDeploy

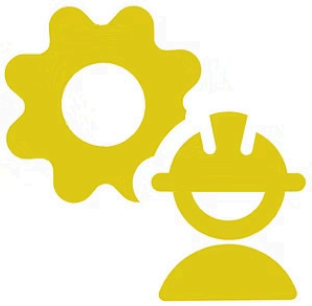
Implanta artefato no ambiente



Para mais exemplos de CodePipeline, acesse: https://aws.amazon.com/pt/blogs/devops/tag/aws-codepipeline/page/3/

AWS DevOps & Developer Productivity Blog: https://aws.amazon.com/pt/blogs/devops/

Pontos Chave do CI/CD AWS



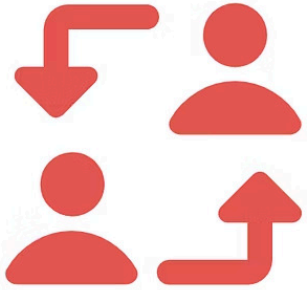
Automação

Automatizam todo o fluxo de entrega



Consistência

Garantem deploys consistentes e ágeis



Integração

Conectam-se perfeitamente com repositórios



Velocidade

Reduzem erros manuais e aceleram entregas

Infraestrutura como Código (IaC)

O Conceito

Gerenciar e provisionar infraestrutura através de código.

Substituir cliques manuais por arquivos de definição.

Benefícios

- Automação
- Consistência
- Replicabilidade
- Versionamento

AWS CloudFormation

Modelagem

Serviço AWS para modelar e provisionar recursos via código.

Templates

Usa YAML ou JSON para definir a infraestrutura (Stack).

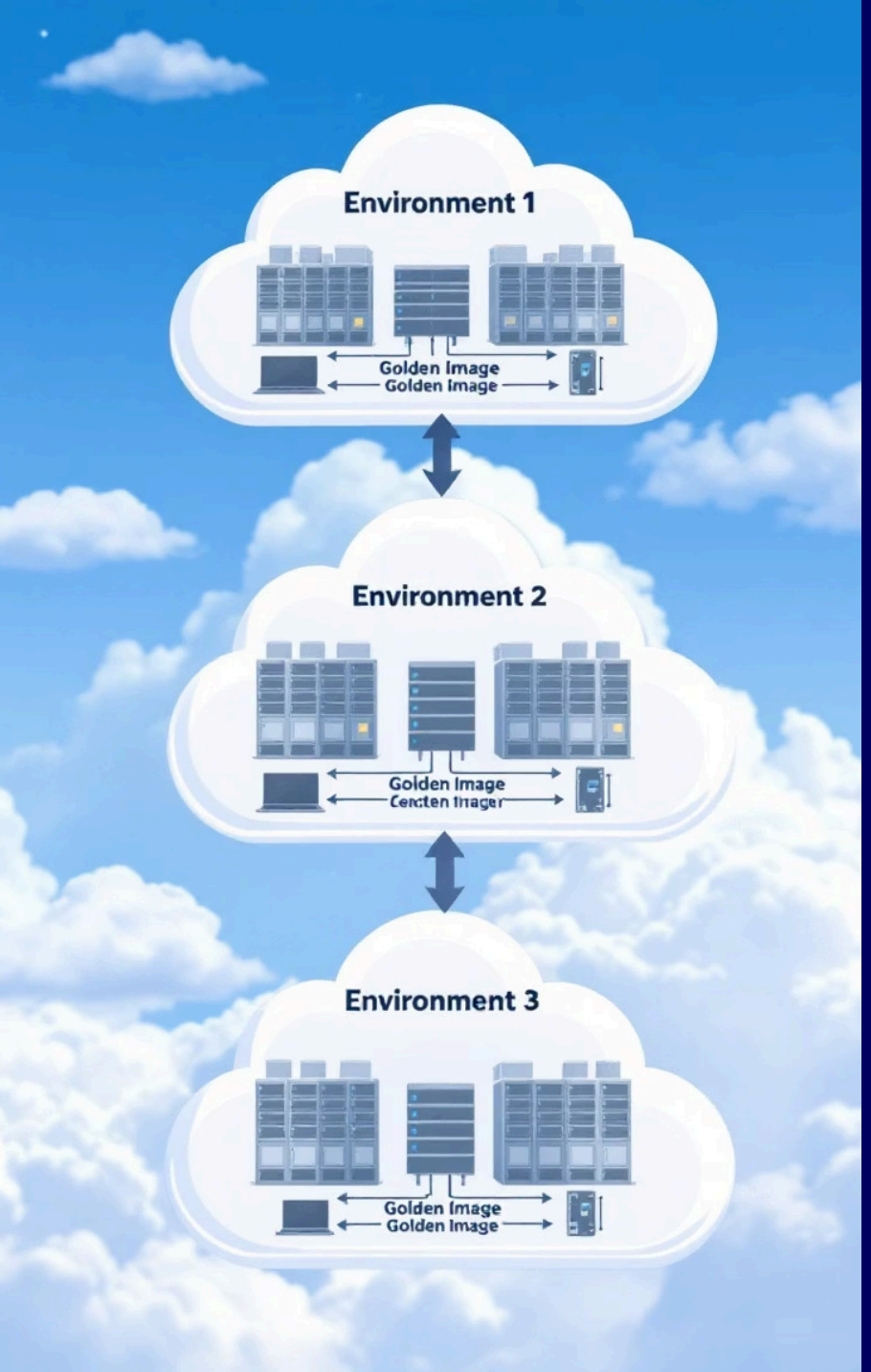
Idempotência

Aplicar o mesmo template múltiplas vezes resulta no mesmo estado.

Gerenciamento

Controla criação, atualização e exclusão de recursos.





Benefícios e Uso do CloudFormation

Consistência

Ambientes idênticos entre Dev, Staging e Prod.

Automação

Provisionamento e atualização automatizados de recursos.

Versionamento

Templates no Git para controle de versão da infraestrutura.

Replicabilidade

Criar novos ambientes facilmente com o mesmo template.

Exemplo de CloudFormation

Este template YAML define um template no CloudFormation que define uma função Lambda e um API Gateway simples.

O CloudFormation criará ou atualizará estes recursos automaticamente.

Referências entre recursos são resolvidas durante o deploy.

```yaml
Resources:
  MyLambdaFunction:
    Type: AWS::Lambda::Function
    Properties:
      FunctionName: MyFunction
      Handler: index.lambda_handler
      Role: arn:aws:iam::123456789012:role/MyLambdaRole
      Runtime: python3.9
      Code:
        ZipFile: |
          def lambda_handler(event, context):
              return {"statusCode": 200, "body": "Hello, world!"}

  MyApi:
    Type: AWS::Serverless::Api
    Properties:
      StageName: Prod
      DefinitionBody:
        openapi: 3.0.1
        info:
          title: MyAPI
          version: "1.0"
        paths:
          /hello:
            get:
              x-amazon-apigateway-integration:
                uri:
                  Fn::Sub: arn:aws:apigateway:${AWS::Region}:lambda:path
                      /2015-03-31/functions/${MyLambdaFunction.Arn}/
                      invocations
                httpMethod: POST
                type: aws_proxy
```

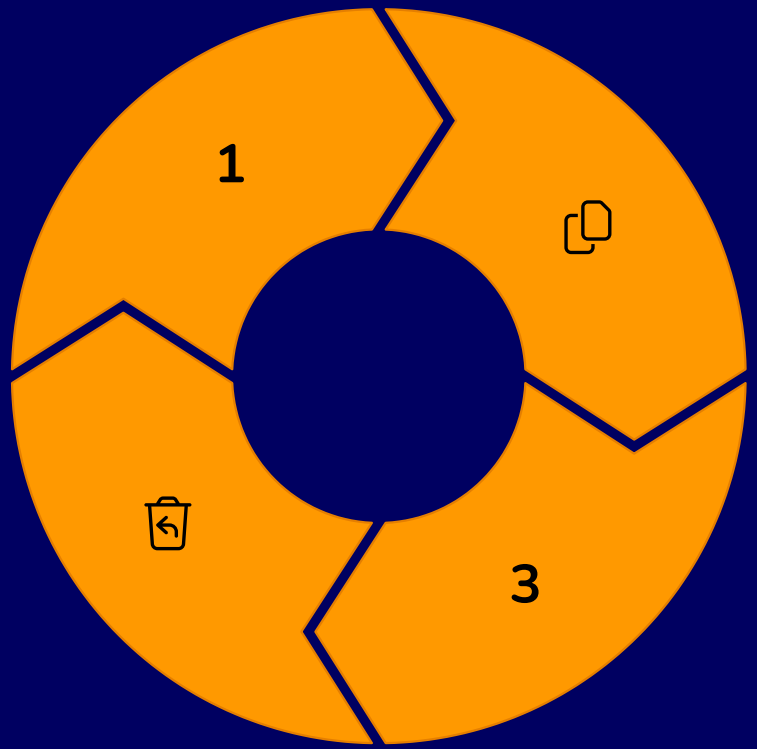
Pontos Chave do CloudFormation

Templates

Cria e gerencia recursos AWS via código

Rollback

Facilita reversão de mudanças problemáticas



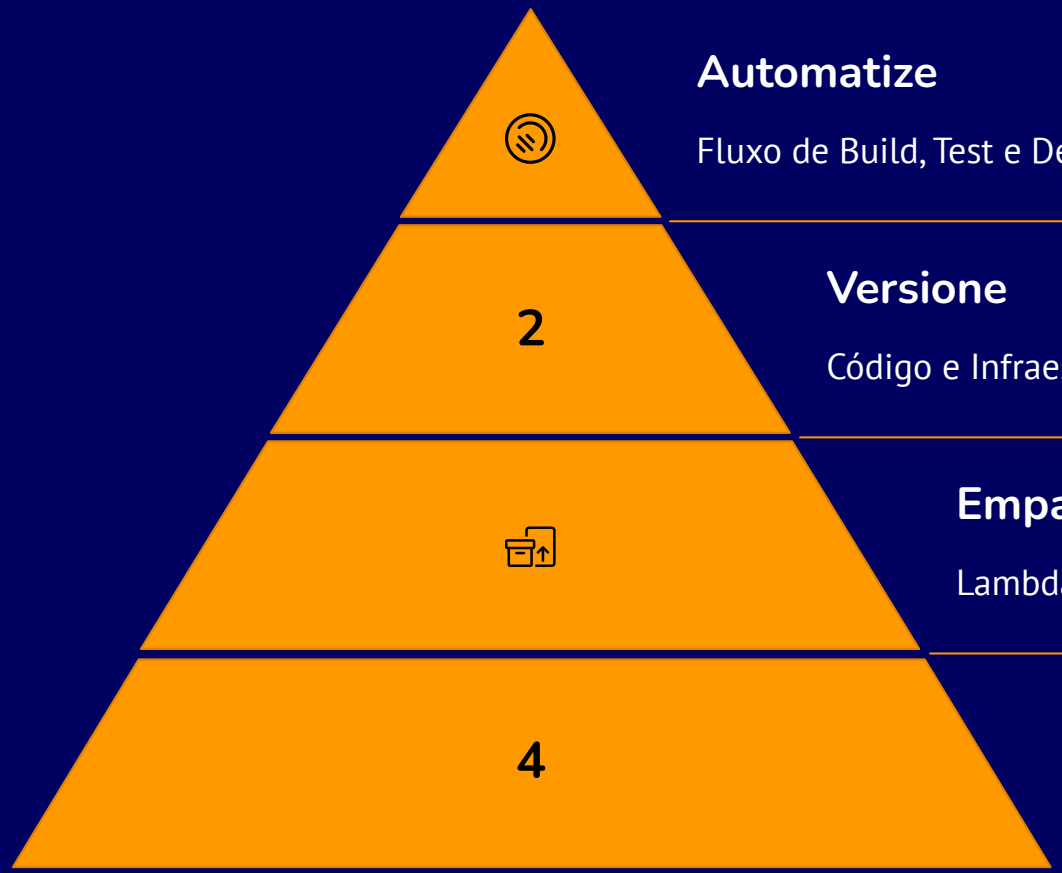
Consistência

Garante ambientes idênticos e replicáveis

Versionamento

Permite controle de versão da infraestrutura

Resumo Final - Pontos Essenciais



Automatize

Fluxo de Build, Test e Deploy

Versione

Código e Infraestrutura

Empacote

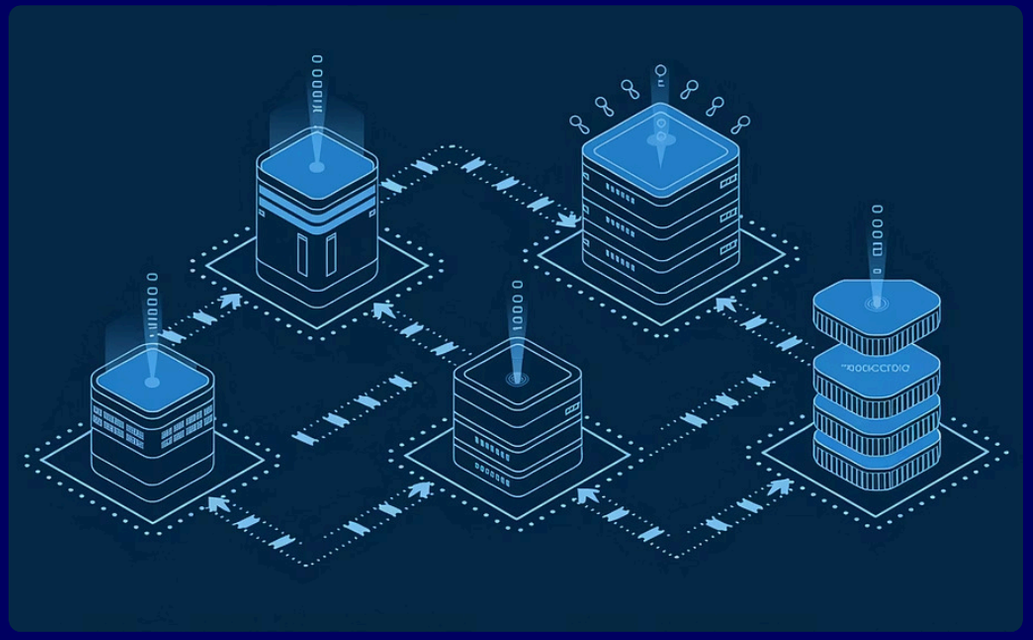
Lambda (ZIP/Layers) e Contêineres

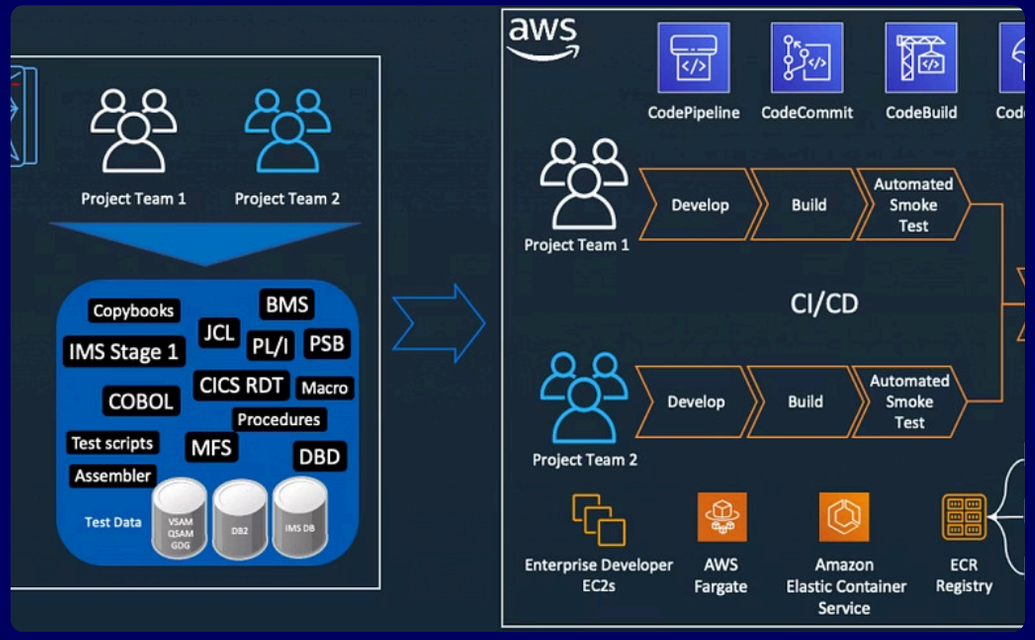
Externalize

Configurações e Segredos

Por Que Isso É Importante

(Visão Prática e da Certificação)





Organização e Automação

Empacotar código e dependências adequadamente torna o deploy simples e confiável.

Reduz erros manuais e facilita manutenção contínua.

Impacto em Produção

Scripts de automação padronizam processos de build.

Contribui para entrega rápida e confiável de novas funcionalidades.

Integração com Ferramentas de Versionamento e Uso de Infraestrutura como Código

Rastreabilidade completa

Ferramentas de versionamento garantem histórico detalhado de alterações no código e infraestrutura.

Consistência garantida

CloudFormation assegura ambientes idênticos em todas as fases do desenvolvimento.

Qualidade elevada

Pipelines de CI/CD automatizam testes e validações, reduzindo drasticamente erros em produção.

Reversibilidade segura

Mudanças problemáticas podem ser facilmente revertidas com controle de versão.

Relevância para Certificação

Conhecimento essencial para obter a certificação AWS Certified Developer – Associate, demonstrando profundo entendimento dos serviços abordados neste módulo.

1 Tópicos de Alta Incidência

Serviços como CodePipeline, CodeBuild e CodeDeploy aparecem frequentemente no exame, com questões que avaliam seu conhecimento sobre integração entre estes componentes.

2 Questões Práticas

O exame apresenta cenários onde você precisa identificar a melhor abordagem para empacotamento de aplicações, configuração de pipelines CI/CD, e implementação de infraestrutura como código.

3 Conceitos Fundamentais

Compreender profundamente CloudFormation, gestão de artefatos, e estratégias de implementação contínua são habilidades diferenciais que aumentam suas chances de aprovação.

Candidatos bem-sucedidos demonstram não apenas conhecimento teórico, mas a capacidade de aplicar estes conceitos em cenários reais de desenvolvimento na nuvem AWS.

Este módulo cobre aproximadamente 25% do conteúdo avaliado no exame, tornando-o um componente crucial da sua preparação para obter a certificação AWS Certified Developer – Associate.